

CampusEats - Capability List

Distinct jobs the system is responsible for (the verbs / capabilities)

Based on the CampusEats brief, the system has six distinct capabilities. Each capability describes a clear job the system performs.

1. Manage profile and addresses	Maintain student accounts, profiles, and delivery addresses.
2. Browse restaurants and menus	Show campus restaurants, menus, and available prices.
3. Add to cart and place order	Manage carts and create orders with their status.
4. Pay online / refund	Process payments and handle refunds.
5. Assign rider and track delivery	Assign deliveries to riders and provide delivery tracking.
6. Send notifications	Send order and delivery status updates to students.

One Operation in Full: placeOrder

The placeOrder operation demonstrates how a complete order placement workflow interacts with the system's services, including input parameters, expected output, and potential error scenarios.

INPUT	OUTPUT	ERRORS
• studentId • items: [[itemId, qty]] • deliveryAddressId • paymentMethodId	• orderId • status: 'placed' • total • estimatedMinutes	• item unavailable • payment declined • invalid address • empty cart

Services Contract Tables

Accounts Service

Operation	Input	Output	Errors
getUser	User reference	User information	User not found
getAddress	User reference, Address reference	Address information	Address not found; Address does not belong to user

Catalogue Services

Operation	Input	Output	Errors
getRestaurant	Restaurant reference	Restaurant information	Restaurant not found
getMenu	Restaurant reference	Available menu items with prices	Restaurant not found; Menu unavailable
checkItem	Restaurant reference, Item reference, Quantity	Item availability and current price	Item not found; Item unavailable; Invalid quantity

Orders Service

Operation	Input	Output	Errors
placeOrder	User reference, Restaurant reference, Selected items, Delivery address, Payment details	Order confirmation, Total amount, Order status	Invalid item; Item unavailable; Invalid address; Payment failed
getOrder	Order reference	Order details, Selected items, Total amount, Current status	Order not found; Unauthorized access

cancelOrder	Order reference, Cancellation reason	Updated order status	Order not found; Order cannot be cancelled; Cancellation not allowed
-------------	---	----------------------	---

Payments Service

Operation	Input	Output	Errors
processPayment	Order reference, Amount, Payment method	Payment status and transaction reference	Payment declined; Invalid payment details; Payment gateway unavailable

Delivery Service

Operation	Input	Output	Errors
assignDelivery	Order reference, Delivery address	Delivery assignment and status	No rider available; Invalid address; Assignment failed

Customer Engagement Service

Operation	Input	Output	Errors
sendNotification	User reference, Event type, Message, Channel	Notification delivery status	Invalid recipient; Unsupported channel; Delivery failed
submitReview	User reference, Order reference, Target, Rating, Comment	Review confirmation	Invalid order; Review not allowed; Invalid rating
searchCatalogue	Search query, Optional filters	Matching restaurants and menu items	Invalid query; Search unavailable

Service Validation & Dependencies

Validation Analysis

Service	Reachable	Self-contained	Has a Contract	Independent	Loosely Coupled
Accounts	Pass	Pass	Pass	Pass	Pass
Catalogue	Pass	Pass	Pass	Pass	Pass
Orders	Pass	Pass	Pass	Partial — depends on Accounts, Catalogue, Payments and Delivery during order processing	Partial — multiple service calls exist; stable contracts/APIs should be maintained
Payments	Pass	Pass	Pass	Pass	Pass
Delivery	Pass	Pass	Pass	Pass	Pass
Customer Engagement	Pass	Pass	Pass	Partial — depends on Accounts, Orders and Catalogue for some operations	Partial — multiple crossservice calls exist; API contracts should remain stable

Remediation Recommendations

Service	Issue	How to Fix
Orders	Independent — Partial	Keep Accounts, Catalogue, Payments and Delivery behind stable service contracts. Avoid direct access to their internal data.
Orders	Loosely Coupled — Partial	Use well-defined API calls/events and exchange only the minimum information required by each operation.
Customer Engagement	Independent — Partial	Keep user/order/catalogue dependencies behind stable contracts so its internal implementation can change independently.

Customer Engagement	Loosely Coupled — Partial	Use service APIs or events instead of directly accessing other services' data.
---------------------	---------------------------	--